

effect-classifier 改善に関する技術選定および検証レポート (SetFit fine-tuning 効果評価)

1 目的と背景

effect-classifier の改善として、(1) モデル差し替え (ruri-v3-30m)、(2) kNN 化、(3) exemplar 拡張 (training-data.json)、(4) SetFit fine-tuning の 4 段階を実施した。

本レポートは (4) fine-tuning の効果を評価し、初回 (v1) の訓練手法に重大な欠陥があったことの分析、ならびに修正版 (v2) による再評価と結論をまとめる。

2 用語と評価観点

2.1 用語 (negative / none / OOS の統一)

本書では「**negative**」を「いずれの効果クラスにも属さない入力 (**OOS: out-of-scope**)」と定義し、学習・分類上は **none** クラス (追加クラス) として扱う。

2.2 評価観点

fine-tuning の価値は、訓練データに含まれない未知表現への **汎化性能** 向上にある。従って、(i) 小規模の手作業テスト (44 件) だけで結論を出さず、(ii) 未知表現を集めた大規模テストで差分を確認する。

3 第 1 回評価 (v1: 欠陥あり fine-tuning)

3.1 方法

44 件のテストケース (positive 38 件 + negative 6 件) に対して 4 構成を比較した。各構成で最適パラメータ (z, minGap) をグリッドサーチし、F1 最大値を比較した。

構成	exemplar 数	最適 F1	Precision	Recall
A: base + EFFECT_PHRASES	84	77.1%	84.4%	71.1%
B: base + training-data	271	86.8%	86.8%	86.8%
C: ft + EFFECT_PHRASES	84	85.7%	84.6%	86.8%
D: ft + training-data	271	86.5%	88.9%	84.2%

改善の要因	$\Delta F1$	計算
exemplar 拡張 (84→271)	+9.7pp	B - A
fine-tuning (exemplar 84)	+8.6pp	C - A
fine-tuning (exemplar 271 の上に)	-0.4pp	D - B

3.2 結果

3.3 解釈上の注意

上記だけから「fine-tuning は不要」と結論づけるのは誤りである。後述の通り、v1 の fine-tuning 手法自体に欠陥があり、fine-tuning というアプローチの可否を正しく評価できていない。

4 v1 fine-tuning の欠陥分析

4.1 欠陥 1: negative 例が訓練に一切使われていない

fine_tune.py にて、negative 例 (OOS) は全て無視されていた：

```
if not ex["is_positive"]:
    continue # 50件の negative 例は全て無視される
```

SetFit の対照学習はクラス間分離を学習するが、「どのクラスにも属さない」を学習しない。結果として無関係語 (例：ありがとう、了解等) が特定クラスに引き寄せられ、false positive の増加要因となる。

4.2 欠陥 2: loss 関数が弱い選択

SetFit デフォルトの CosineSimilarityLoss は、他の loss と比べ学習シグナルが弱いことが示されている。また、意味的に近い別クラス (heat vs light 等) と無関係ペア (heat vs greeting 等) の難易度差を十分に反映しにくい。

代替候補：

- CoSENTLoss: CosineSimilarityLoss の上位互換として位置づけられる
- MultipleNegativesRankingLoss (MNRL): in-batch negatives を効率的に利用

4.3 欠陥 3: 過学習／次元崩壊の兆候

fine-tuning 後にスコア分布が激変しており、embedding 空間が大きく変形している。validation なしで全データを学習している点、ならびに num_iterations=20 により大量ペアが生成される点から、過学習や次元崩壊（有効次元数の低下）が疑われる。

4.4 欠陥 4: 評価方法が汎化性能を測れていない

44 件の手作業テストはサンプルサイズが小さく、手選びバイアスも入り得る。fine-tuning の本来の価値（未知表現への汎化）を測るには不十分である。

5 修正アプローチ (v2)

5.1 訓練の修正方針

項目	v1 (欠陥あり)	v2 (修正版)
negative 例	訓練から除外 (0 件)	none クラスとして 45 件を追加
loss 関数	CosineSimilarityLoss	CoSENTLoss
num_iterations	20	8
batch_size	8	16
validation	なし	20% stratified split (accuracy=83.6%)

5.2 none クラスの構成

none クラス (OOS) は以下のカテゴリから多様な例を収集した：挨拶・社交表現、ゲーム操作発話、日常会話、感嘆詞、方向指示、質問／確認等。

6 第 2 回評価 (v2: 修正版 fine-tuning, 44 件)

6.1 スコア分布の変化

	base	v1 ft	v2 ft
mean	0.8691	0.5432	0.6618
std	0.0186	0.1419	0.0787

v2 では分布の過度な変形が緩和され、v1 で観測された次元崩壊の兆候は軽減された。

構成	最適 F1	v1 との差
A: base + EFFECT_PHRASES	77.1%	(同一)
B: base + training-data	86.8%	(同一)
C: ft + EFFECT_PHRASES	87.2%	+1.5pp
D: ft + training-data	87.2%	+0.7pp

改善の要因	v1 Δ F1	v2 Δ F1
exemplar 拡張 (B - A)	+9.7pp	+9.7pp
fine-tuning (C - A)	+8.6pp	+10.0pp
fine-tuning の追加価値 (D - B)	-0.4pp	+0.3pp

6.2 4 構成比較結果

6.3 所見

小規模 (44 件) では、fine-tuning の追加価値 (D-B) は +0.3pp と微小であり、差は安定して観測されにくい。

7 第 3 回評価 (大規模汎化性能テスト: 156 件の未知表現)

7.1 方法

training-data.json にも EFFECT_PHRASES にも含まれない 156 件の未知表現でテストした。構成は base / fine-tuned v2 の両方で training-data (271 exemplar) を使用し、実運用と同条件とした。

7.2 結果

	base model	fine-tuned v2
最適パラメータ	$z = 0.3, \text{minGap}=0.01$	$z = 0.3, \text{minGap}=0.01$
Precision	79.2%	78.6%
Recall	81.7%	87.3%
F1	80.5%	82.7%
TP	103	110
FP	27	30
FN	23	16
TN	3	0

本条件では Δ F1 = +2.2pp (改善 15 件、悪化 11 件、net +4 件) となり、未知表現への汎化性能改善を確認した。一方で TN が 3→0 であり、OOS (none) 棄却は依然として課題である。

8 総合結論（技術選定）

1. exemplar 拡張は最も効果的（+9.7pp）で、低コストかつ確実である。
2. fine-tuning v2 は追加的な改善を提供する。小規模（44 件）では差が見えにくい、大規模未知表現（156 件）で F1 +2.2pp を確認した。
3. ただし、Recall 改善と Precision 微減のトレードオフがあり、OOS 棄却（none）精度は別途の対処が必要である（運用閾値 z の設計、none 例の増強等）。
4. 導入コスト（モデルサイズ、推論パイプライン、閾値管理）に対し、+2.2pp を許容できるかはプロジェクト方針次第である。

9 残課題

- 実ユーザーの音声認識データでの評価（現状はシミュレーション中心）
- negative / none 例の質と量の強化による OOS 棄却精度の改善
- friction_reduce / electric の悪化への対処（属性間境界の改善）

10 再現手順

評価の再現

```
cd server
```

```
npx tsx test/generalization-test.ts
```

第3回：156件汎化性能テスト

```
npx tsx test/production-comparison.ts
```

第2回：4構成比較（44件）

```
npx tsx test/finetune-analysis.ts
```

同一条件詳細分析（EFFECT_PHRASES のみ）

```
npx tsx --test test/effect-classifier.test.ts # 通常テスト
```

v2 fine-tuning の再訓練

```
cd training
```

```
uv run python fine_tune.py
```

```
uv run python export_onnx.py
```

11 参考文献

- SetFit: Efficient Few-Shot Learning Without Prompts (arXiv:2209.11055)
- sentence-transformers Loss Overview (sbert.net)
- Understanding Dimensional Collapse in Contrastive Self-supervised Learning (ICLR 2022)
- DETER: Improved Out-of-Scope Intent Classification (LREC-COLING 2024)